

# Write-Public Oblivious RAM and Its Applications

Ananya Appan<sup>1\*</sup>, James Austgen<sup>2\*</sup>, Kushal Babel<sup>3</sup>, and Sourav Das<sup>3</sup>  
<sup>1</sup>UIUC, <sup>2</sup>Cornell University, <sup>3</sup>Category Labs.

## Abstract

Oblivious RAM (ORAM) protocols enable a resource-constrained client to outsource data to an untrusted server, and access this data without revealing what is being accessed. While ORAM protocols are almost optimal, concretely efficient ORAM protocols incur sub-optimal bandwidth overhead and require multiple roundtrips of communication between the client and the server.

In this work, we introduce *write-public ORAM*: a relaxed variant of ORAM that hides what data is being read, but not what data is being updated on the server. We present *wpORAM*, a write-public ORAM that modifies Path ORAM to achieve a  $\log N$ -factor improvement in bandwidth and roundtrips over Path ORAM *during writes*. The asymptotic cost of reads in *wpORAM* remains the same as Path ORAM, with a slight slowdown in practice. *wpORAM*, when used with a Trusted Execution Environment (TEE), can implement a dynamic Private Information Retrieval scheme that efficiently updates data stored in a public database.

We use *wpORAM* to implement a private blockchain RPC server that enables blockchain clients to query RPC nodes without revealing their queries to the server operator. Our evaluation shows that, while syncing with the latest blockchain state, our RPC node achieves a 3–4 $\times$  speedup over Path ORAM. Our RPC node can serve reads with a slowdown of approximately 23% compared to Path ORAM.

## 1 Introduction

Oblivious RAM (ORAM) [17] allows a client to outsource data to an untrusted server and access it without revealing which data it accesses. After years of effort [6, 7, 17, 18, 19, 20, 26, 28, 29, 32], ORAM is nearly optimal, but it is still expensive. Any ORAM must communicate  $\Omega(\log N)$  words per access [21] for a server holding  $N$  words, and every ORAM we are aware of incurs  $\Theta(\log N)$  roundtrips. Path ORAM [29],

the most efficient ORAM in practice, pays a further  $\log N$  factor in bandwidth when the word size is  $\Theta(\log N)$ , and the construction that meets it [7] has very large hidden constants.

In this paper, we introduce *write-public ORAM*, a relaxation of ORAM that hides the access pattern of reads but not the access pattern of writes. For each access, a write-public ORAM reveals whether the access is a read or a write, and, if it is a write, the address being written to. Everything else stays hidden, including the addresses that are read and the contents that are read and written. Many applications need nothing more. When a client reads from a public database, updates to that database are public already, and only the client’s reads must be kept private.

Relaxing ORAM to gain efficiency is not new. Various works have designed oblivious algorithms for special-purpose RAM programs, specifically pointer programs [4, 24, 33], or more constrained adversaries where the adversary can not observe physical read locations [9, 22, 27]. These works reduce bandwidth and roundtrip costs by a factor of  $\Theta(\log N)$  compared to generic ORAM.

**Motivation for write-public ORAM.** We see write-public ORAM as an efficient alternative to private information retrieval (PIR) schemes that *must* support efficient updates. PIR [14] lets clients fetch entries of a public database without revealing which entry they fetch. PIR schemes come with a fundamental cost: the server must touch every record on each query. One way to lower this cost is to store the database within a TEE, where a client’s query remains hidden even from the operating system. TEEs, however, provide limited secure storage, which makes this approach unusable.

Pairing the TEE with an ORAM removes this obstacle. The TEE stores an encrypted copy of the database in untrusted storage and accesses it through an ORAM, which hides which records each query touches. As a result, each query costs only a polylogarithmic number of accesses to untrusted storage. Moreover, when the ORAM is write-public, this approach can efficiently support public updates to the database.

The gains from write-public ORAM increase with the write-heavy nature of the workload. Efficient writes are an impor-

---

\*Equal Contribution. Work done while at Category Labs.

| Metric     | Path ORAM [29]                 | OptORAMa [7]        | wpORAM              |                                |
|------------|--------------------------------|---------------------|---------------------|--------------------------------|
|            |                                |                     | Write               | Read                           |
| Bandwidth  | $O(\log N \cdot w + \log^3 N)$ | $O(\log N \cdot w)$ | $O(\log N \cdot w)$ | $O(\log N \cdot w + \log^3 N)$ |
| Roundtrips | $O(\log N)$                    | $O(\log N)$         | $O(1)$              | $O(\log N)$                    |

Table 1: Comparison of bandwidth (bits) and roundtrips with state-of-the-art ORAMs.  $w$  denotes the word size.

tant property of many PIR schemes [2, 5, 13, 23]. Two such applications are privately querying certificate transparency logs and, as we discuss later, enabling clients to query public blockchain state privately [10].

**A construction of write-public ORAM.** We formalize write-public ORAM in Section 4, and design wpORAM<sup>1</sup>, a construction that reduces the roundtrips per write by a  $\log N$  factor over both Path ORAM [29] and OptORAMa [7]. wpORAM also reduces the bandwidth per write by a  $\log N$  factor over Path ORAM. Reads in wpORAM cost the same as reads in Path ORAM, in both bandwidth and roundtrips. We compare these three constructions in Table 1.

wpORAM builds on Path ORAM and inherits its concrete efficiency. Path ORAM maps each logical address to a uniformly random leaf of a binary tree, and keeps this mapping in a *position map* that is itself stored recursively as a smaller ORAM. This recursion is what costs a  $\log N$  factor in bandwidth and roundtrips. Since write addresses are public in our setting, wpORAM serves writes from a separate, *non-oblivious* position map that needs no recursion, while reads keep using the recursive oblivious position map and stay oblivious. Most of our technical work goes into keeping the two position maps consistent without giving up the efficiency of writes. A read remaps its address to a fresh leaf, and this new mapping must reach the non-oblivious position map without revealing which address it belongs to. We meet this requirement by implementing the non-oblivious position map as a *read-public* ORAM, the mirror image of what we set out to build, and we give a simple construction of one.

**A private RPC server for blockchains.** A public blockchain publishes its entire history for anyone to read, but reading it directly requires running a blockchain node, which is expensive. Most users rely instead on a third-party remote procedure call (RPC) provider, which answers queries about balances, transactions, and contract state on their behalf. These queries are sensitive: they often contain addresses and other parameters that leak the precise identity and activity of the user. For example, wallet software commonly simulates transactions prior to signing them; these requests could be used to predict the transaction a user is about to make. The RPC provider also learns the IP address of the sender and when the sender is active, which facilitates cross-account tracking.

Users today have to trust providers not to log or act on these queries. RPC server listings such as Chainlist [11] publish

each provider’s stated privacy policy. However, users have no means of verifying whether RPC providers actually follow these policies. This has motivated work on *private RPC servers*, in which the provider learns nothing about the queries clients make [8, 30].

A private RPC server is a natural fit for wpORAM. Every write to a public blockchain is public already, so only reads have to be hidden. Writes are frequent: recent blockchains produce a block every few seconds or faster, and the server must keep up with the chain before it can serve any reads. Reads are not all simple balance lookups either. A request that runs a smart contract to read its state issues a sequence of dependent reads. Answering such a request over a network would cost a roundtrip per read, which is why we place the RPC logic inside a TEE.

**Implementation and evaluation.** We implement a private RPC server for an Ethereum execution client on top of wpORAM, and compare it against the same client backed by Path ORAM. Our evaluation illustrates that wpORAM improves write throughput by 3–4 $\times$ , at the cost of only a 23% reduction in read throughput. To our knowledge, this is the first implementation of a private RPC server.

**Paper organization.** The rest of the paper is organized as follows. We present preliminaries, including a review of Path ORAM, in Section 2, and an overview of wpORAM in Section 3. We define write-public ORAM, read-public ORAM, and the other oblivious machines we use in Section 4. We present our wpORAM construction in Section 5, and the read-public ORAM that it uses as a building block in Section 6. We describe our private RPC server in Section 7 and report our evaluation results in Section 8. We present additional optimizations in Section 9, and compare with related work in Section 10.

## 2 Preliminaries

### 2.1 Threat Model

For our write-public ORAM protocol, we assume that the server is untrusted but semi-honest. This means that the server faithfully follows the protocol, but is interested in learning the client’s logical address access pattern during reads. We assume that the server can observe *physical* reads and writes performed on server storage to try to learn this access pattern. While we focus on the case when the server is semi-honest for simplicity, since our construction builds on Path ORAM,

<sup>1</sup>Pronounced as “whoop ORAM.”

we can also handle a malicious server by using techniques developed for Path ORAM [25].

Our write-public ORAM completely hides the access pattern of read requests. We remark that all data-content is encrypted and kept hidden from the server, even during writes. Section 4 concretely defines the security guarantees of our write-public ORAM protocol.

For our private RPC application, we assume a Trusted Execution Environment (TEE). We additionally assume that all memory accesses within the TEE are completely hidden from the adversary.

## 2.2 A Review of Path ORAM

Since wpORAM builds on top of Path ORAM, we review the Path ORAM construction [29]. Recall that the client outsources some (encrypted) data of size  $N$  to an untrusted server. Just like RAM, the ORAM interface allows the client to access an address that identifies a value in server storage. To hide this address, the ORAM client maps the address to a uniformly random physical location on the server, which it uses to store the (encrypted) value written to that address. But if the client repeatedly accesses this mapped location, the server may analyze the client’s access pattern to infer which data were accessed [17]. To prevent this, the client continuously *shuffles* server storage by re-mapping logical addresses to new physical addresses each time they are accessed. This is done without revealing to the server how the data is shuffled. We next elaborate on how this shuffling is done.

We start by explaining how server storage is organized. In Path ORAM, each value is written to a binary tree with  $N$  leaves. Each node in this tree holds a bucket that can store a constant (parametrized) number of data-blocks, each of which stores the value assigned to an address along with additional meta-data (such as the leaf the address is mapped to). Path ORAM maintains the invariant that each logical memory address is assigned a uniformly random leaf known only to the client and that, on the server, the value written to that logical address lies somewhere on the path to this leaf. For simplicity, suppose that the client has local storage of size  $N$  that allows it to store a mapping between addresses and leaves (we will later discuss how to remove this requirement). This mapping is called a *position map*.

To access a logical address, the client reads the position map to find which leaf the address is mapped to, downloads the path to this leaf from the server, and scans the path to find the desired data-block containing its value. This leaks nothing about the address being accessed, because each leaf is chosen uniformly at random. Since this path is read in the clear, the client re-assigns the address to a new uniformly random leaf; this allows the client to “shuffle” server-side data. The client then writes this data-block back to the root of the tree<sup>2</sup>, which

maintains the invariant that the block lies on the path to the newly assigned leaf. Based on whether the access is a read or a write, the client either writes back the same data-block or an updated data-block.

All data-blocks cannot remain in the root, since it can only store a constant number of data-blocks. To make space for more data-blocks, the client periodically runs an *eviction* procedure that moves data-blocks towards their respective assigned leaves. During eviction, the client downloads a path, locally re-organizes it to push data-blocks as far as possible towards their assigned leaves (while respecting bucket size constraints), and uploads the path to the server. Path ORAM evicts paths in a deterministic reverse-lexicographic order<sup>3</sup>, which ensures that evictions leak nothing about the addresses that the client reads/writes.

However, this solution requires the client to store a large position map. Path ORAM instead *recursively* implements the position map using a smaller ORAM with  $N/2$  leaves, which is only half the size of the data tree. It is important that the position map is of half the size so that the recursion ends quickly. When the recursion unfolds, the position map is stored in ORAMs of size  $N/2$ ,  $N/4$ ,  $N/8$ , and so on, until the smallest position map is stored locally on the client, which ultimately keeps the position map hidden from the server. Since the position map is also implemented as an ORAM, the client can retrieve the leaf associated with a given address without revealing to the server which address is being queried.

To halve the size of the position map, the client stores *two* address-to-leaf mappings in a *single* data-block: the leaves for addresses 0 and 1 are stored in a single data-block, the leaves for addresses 2 and 3 are stored in another data-block, and so on. We refer the interested reader to [28] for more details.

In sum, Path ORAM stores data blocks along uniformly random paths in a tree, while maintaining the mapping from logical addresses to leaves in a recursively implemented position map. Reading a path from the data-tree incurs  $O(\log N \cdot w)$  bandwidth, where  $w$  is the word size of values stored in memory. Looking up the position map incurs  $O(\log^3 N)$  bandwidth, because (1) the position map is recursively implemented using  $\log N$  ORAMs, (2) the client must download a path of size  $\log N$  from each ORAM, and (3) each ORAM stores addresses of size  $\log N$ . Note that each of these  $\log N$  paths must be downloaded *sequentially*, starting from the smallest ORAM, since the address-to-leaf mapping stored in one ORAM is used to determine the path to be read in the next ORAM. This incurs  $O(\log N)$  roundtrips.

<sup>2</sup>Strictly speaking, the client writes the block to a local data structure called the Stash. We will explain more about this in Section 6. For ease of

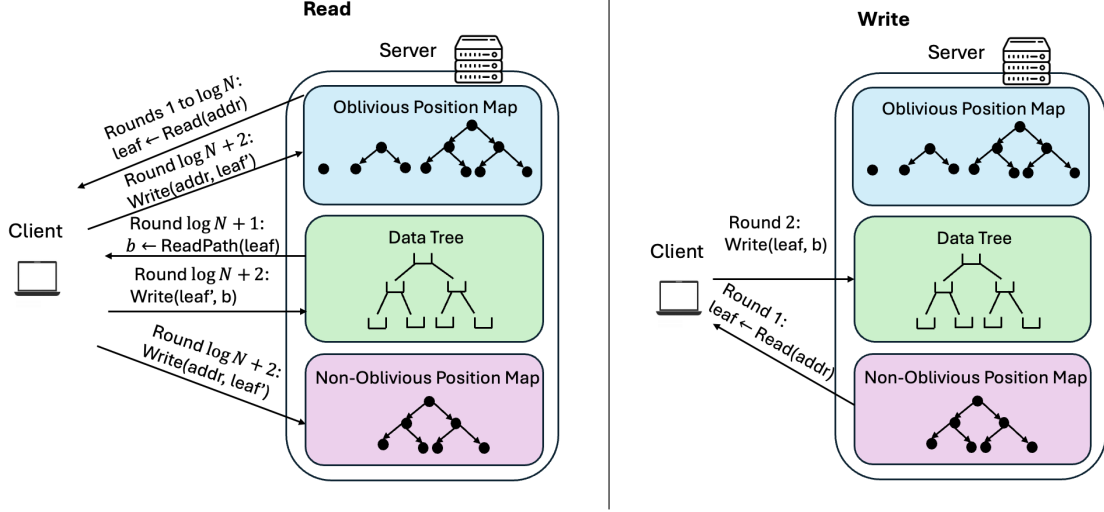


Figure 1: An illustration of how wpORAM handles reads and writes, respectively.

### 3 Overview of wpORAM

Path ORAM hides the address accessed during reads and writes, and it additionally hides whether the access is a read or a write. However, not all applications require writes to be private; for example, a client might require oblivious read access to public, frequently updated data. When writes are “public”, meaning that the address used during a write and the type of access (read/write) need not be hidden, wpORAM reduces the bandwidth and roundtrips incurred during writes by a  $\log N$  factor.

wpORAM is a modification of Path ORAM. The primary challenge is that the addresses used during reads must remain hidden. As a result, even while writing to an address, the value written to an address must be encrypted and written on the path to a uniformly random leaf that is hidden from the server. While we do not change the structure and invariants of Path ORAM’s binary data tree, we observe that modifying the use of the *position map* yields improved performance.

**Observation 1: Writes do not need a recursive position map.** When we perform public writes, we reveal the address being written to. As a result, during writes, the client can use a position map that does not hide the address being looked up. Note that the leaf mapped to that address must still be kept private. This is because the read access pattern must remain hidden. In particular, when an address is read after being written to, the path that the client downloads should not reveal the address.

Our construction uses *two* position maps: an oblivious one that hides the address being looked up during reads, and a

explanation, here we will assume that the client instead writes the data-block to the root.

<sup>3</sup>Reverse-lexicographic order sorts numbers by the *reverse* of their bit-wise representation. For instance, when considering numbers from 0 to 7, the reverse-lexicographic ordering is 0, 4, 2, 6, 1, 5, 3, 7.

*non-oblivious position map* that *reveals* the address being looked up during writes, but hides the leaf that the address is mapped to. The non-oblivious position map does not have to be implemented recursively using smaller ORAMs, allowing us to gain the  $\log N$  factor improvement! We will shortly describe how we implement the non-oblivious position map. For now, suppose that this is implemented as any (non-oblivious) key-value map stored on the server that maps addresses to encrypted leaves.

Recall that, during a write, Path ORAM remaps the address to a new uniformly random leaf. For *consistency*, both position maps must always store the same address-to-leaf mappings. If the address is re-mapped to a new leaf during a write, the oblivious position map would also have to be updated with the new leaf. This would once again invoke the recursion, resulting in no improvement over Path ORAM. Our next two observations focus on how we can maintain consistency between the two position maps, while retaining the  $\log N$  factor improvement.

**Observation 2: Writes do not require a leaf remap.** Recent work [24] shows that an address can be written to<sup>4</sup> the data tree *without remapping* the address to a new leaf. To understand why, observe that values are written back to the data tree via evictions, and that the server-side locations accessed during these evictions are independent of the leaves that their addresses are assigned to. To write a value to an address, the client can (1) look up the non-oblivious position map to see which leaf it is mapped to, (2) write the (encrypted) data-block containing the value (along with the mapped leaf<sup>5</sup>) to the root, and (3) evict *deterministic* paths to move this block towards its mapped leaf. This *does not* reveal the mapped leaf to the

<sup>4</sup>Reads will still require the leaf to be remapped.

<sup>5</sup>the mapped leaf must be written in the data-block to ensure that the data-block moves down the right path during evictions

server, and does not require the address to be mapped to a new leaf.

Note that a path may contain *multiple* versions of the value written to an address, but the client knows that the latest version is in the data-block closest to the root. As evictions happen, the client will replace older versions of the value with newer ones.

To see how this is different from Path ORAM, recall that during both reads and writes, the Path ORAM client downloads the path to the leaf that the address is mapped to. This reveals the leaf to the server, and requires the leaf to be mapped to a new address. Alternatively, in wpORAM, the client downloads the path to the address’s mapped leaf only during a read, and not during a write. During a write, the client only evicts a deterministically chosen path. This eliminates the need for the address to be remapped to a new leaf.

Reads in wpORAM are performed similar to Path ORAM: the client looks up the oblivious position map to find the leaf mapped to a logical address, downloads the path to this mapped leaf from the data tree, and searches this path to find the value written to this address. Since the path to the mapped leaf is revealed to the server, wpORAM requires the leaf to be *re-mapped during reads*. At this time, for consistency, the client must update the address-to-leaf mapping in both position maps, the oblivious position map and the non-oblivious position map, with the new leaf. This must be done without revealing the logical address being re-mapped. While this is straightforward for the oblivious position map, which is also an ORAM, this is trickier for the non-oblivious position map. Because of this, we cannot use a simple non-oblivious key-value map to implement the non-oblivious position map. However, to maintain our  $\log N$  factor improvement for writes, we must ensure that the non-oblivious position map can be accessed at a cheaper cost than the oblivious position map. Our reads are as expensive as Path ORAM, so we don’t need to worry about  $\log N$  factor for reads.

**Observation 3: A read-public ORAM can implement the non-oblivious position map.** Observe that, in the non-oblivious position map, the client must update the leaf associated with an address without revealing the address, whereas it may reveal the address when looking up the leaf associated with that address. In other words, the non-oblivious position map can be implemented using an ORAM that hides the write access pattern but not the read access pattern! This is, in a way, the “reverse” of what we are trying to achieve. But it turns out that designing such a *read-public* ORAM is relatively simple and, indeed, a construction inspired by prior work [24] can be used to achieve this.

In short, such an ORAM can be implemented simply using Path ORAM, but without the position map. Each address is mapped to a *static* leaf that never gets remapped. This static mapping can be done simply by mapping the  $i$ th address to the  $i$ th leaf. The read/write operations can be performed just like in Path ORAM: the client reads an address  $\text{addr}$  by fetching

the path to leaf  $\text{addr}$ , and writes to an address by writing the data-block with the updated value to the root and evicting deterministic paths. Following our discussion for **observation 2**, this reveals the address being accessed during reads, but hides the address being written to during writes.

In sum, wpORAM uses a data tree with the same invariant and structure as Path ORAM, and uses two position maps, an oblivious position map and a non-oblivious position map. When writing to an address, the client reads the *non-oblivious position map* to fetch the leaf mapped to the address, and writes the value (along with this leaf) back to the server via evictions. When reading from an address, the client reads the *oblivious position-map* to fetch the leaf mapped to the address, and reads the path to the mapped leaf in the data tree to retrieve the required value. The client then remaps the address to a new leaf, and obviously updates both position maps with this new address-to-leaf mapping. Figure 1 illustrates.

The non-oblivious position map avoids recursion and is more efficient than the oblivious position map by a factor of  $\log N$ , in both bandwidth and round complexity. Writes access only the non-oblivious position map and the data tree, and thus inherit this  $\log N$  factor improvement. Reads access all three—the data tree, the oblivious position map and the non-oblivious position map—and maintain the same asymptotic efficiency of Path ORAM’s reads, while being only slightly more expensive in practice. We present our construction in detail in Section 6.

## 4 Definitions

Each component of our solution—the data tree, the oblivious position map, and the non-oblivious position map—is implemented using different oblivious *machines* that implement addressable storage. In this section, we describe the interface provided by non-oblivious versions of these machines, and then describe the additional guarantees provided by oblivious versions. We start by defining the interface provided by a (non-oblivious) Random Access Machine.

**Definition 1 (RAM).** A *Random Access Machine (RAM)* implements addressable storage of size  $N$  that handles the following requests.

- $\text{Write}(\text{addr}, v)$ : Writes a value  $v$  to address  $\text{addr}$ .
- $v \leftarrow \text{Read}(\text{addr})$ : Reads the value at address  $\text{addr}$ , if any, and otherwise returns `None`.

A *RAM program* is a program that issues requests to a *Random Access Machine*.

An *oblivious RAM (ORAM)* [17] protocol transforms a RAM program’s request sequence into a sequence of accesses that preserves correctness, while concealing the addresses accessed during both reads and writes. In this work, we implement a *write-public ORAM*, wpORAM, that instead reveals

addresses written to during writes. To implement wpORAM, we also use a *read-public ORAM*, which provides the opposite guarantee of revealing the addresses accessed during reads.

We next define each of these notions. Intuitively, each notion of ORAM ensures that the transformed sequence of requests output by an ORAM only leaks what is allowed by a leakage function  $\mathcal{L}$ . The difference in privacy guarantees is captured by different leakage functions  $\mathcal{L}$  for each ORAM, based on whether the access sequence during reads and writes is public or private.

**Definition 2** (Write-public ORAM/ORAM/Read-public ORAM). Consider a program  $\Pi$  that transforms any sequence of requests  $\vec{r}$ , generated by a RAM program, into a sequence of RAM requests  $\vec{r}'$  such that (1)  $\vec{r}'$  implements a RAM (Definition 1) for  $\vec{r}$  and (2) there exists a polynomial-time simulator  $\mathcal{S}$  for which the following ensembles are statistically close in  $\lambda$ , where  $\lambda$  is the statistical security parameter<sup>6</sup>:

$$\vec{r}' \stackrel{s}{=} \mathcal{S}(1^\lambda, \mathcal{L}(\vec{r}))$$

- **Write-public ORAM:**  $\Pi$  is a write-public ORAM if  $\mathcal{L}(\vec{r})$  leaks, for each request  $r \in \vec{r}$ , (1) whether  $r$  is a Read or a Write and (2) if  $r$  is a Write, the address  $\text{addr}$  that is being written to.
- **ORAM:**  $\Pi$  is an ORAM if  $\mathcal{L}(\vec{r})$  only leaks the number  $|\vec{r}|$  of requests in  $\vec{r}$ .
- **Read-public ORAM:**  $\Pi$  is a read-public ORAM if  $\mathcal{L}(\vec{r})$  leaks, for each request  $r \in \vec{r}$ , (1) whether  $r$  is a Read or a Write and (2) if  $r$  is a Read, the address  $\text{addr}$  that is being read.

The above notions of ORAM are used to instantiate different components of our wpORAM construction. Like Path ORAM, we implement the oblivious position map using an ORAM. As mentioned in the overview, we implement the non-oblivious position map using a read-public ORAM. We will describe how we design a read-public ORAM in Section 5.

We implement the data tree using an oblivious machine from prior work called an *Oblivious Single Access Machine (OSAM)* [4, 24]. Recall that, while writing to an address in wpORAM, the client does not re-map the leaf mapped to that address. Instead, the client writes the updated version of the value to the same path that the address was mapped to previously. But, while reading a logical address, the client must re-map the address to a new leaf since the path to the leaf is revealed in the clear. Hence, the client can “repeatedly write” values on the path to a leaf, but can read the path to a leaf only once. An OSAM helps model this constraint.

We first define the interface of a (non-oblivious) *Single Access Machine (SAM)*. A SAM is a *restrictive* version of RAM that allows the same address (which, for our purposes, can be thought of as a leaf in the data tree) to be written to multiple times, but restricts it to be read at most once.

**Definition 3** (SAM). A *Single Access Machine (SAM)* implements an addressable memory of size  $N$  that handles the following requests.

- $\text{addr} \leftarrow \text{Alloc}()$ : Allocates a fresh address that has not been allocated before.
- $\text{Write}(\text{addr}, v)$ : If  $\text{addr}$  was not allocated, the machine outputs  $\perp$  and halts. Otherwise, the machine writes  $v$  to address  $\text{addr}$ .
- $v \leftarrow \text{Read}(\text{addr})$ : If  $\text{addr}$  was not allocated or was already read from, the machine outputs  $\perp$  and halts. Otherwise, the machine returns the value written to  $\text{addr}$ , or None if no such value exists.

A SAM program is a program that issues requests to a Single Access Machine. A SAM program is **valid** if it never causes the machine to output  $\perp$ .

Notice that a valid SAM program halts if the same address is read twice. We remark that, when compared to RAM, SAM requires an extra  $\text{Alloc}()$  request to enforce this single-read constraint. Each time the client reads an address to fetch a value, the client must reassign and write the value to a *freshly* allocated address to fetch the value again in the future.

An *oblivious SAM (OSAM)* [4, 24] transforms a valid SAM program’s (a SAM program that reads each address at most once and only after it is allocated) request sequence into a sequence of accesses that preserves correctness, while concealing the addresses accessed during both reads and writes.

**Definition 4** (OSAM). Consider a program  $\Pi$  that transforms a sequence of requests  $\vec{r}$ , generated by a valid SAM program, into a sequence of RAM requests  $\vec{r}'$ , such that (1)  $\vec{r}'$  implements a SAM for  $\vec{r}$  and (2) there exists a polynomial-time simulator  $\mathcal{S}$  for which the following ensembles are statistically close in  $\lambda$ :

$$\vec{r}' \stackrel{s}{=} \mathcal{S}(1^\lambda, \mathcal{L}(\vec{r}))$$

$\Pi$  is an OSAM if  $\mathcal{L}(\vec{r})$  leaks, for each request  $r \in \vec{r}$ , the number of requests in  $\vec{r}$  that are either Reads or Writes.

The work of [24] presents Path OSAM, which is essentially Path ORAM, but used without the position map. As discussed previously, the insight behind this is that while a read in Path ORAM reveals the address’s assigned leaf to the server, a write can be performed using deterministic evictions, without revealing this leaf. We use this same observation and construction to implement a read-public ORAM in Section 6.

## 5 Construction of wpORAM

Algorithm 1 presents our wpORAM construction. We gave an overview of our construction in Section 3, where we described our solution as a modification of Path ORAM. Here,

<sup>6</sup>This means the probability of error is at most  $2^{-\lambda}$ .

---

**Algorithm 1:** Code run by the wpORAM client

---

```
Initialize
for  $\text{addr} \in [0, N]$  do
   $\text{leaf} \leftarrow \text{Alloc}()$ 
   $\text{DataTree.Write}(\text{leaf}, \text{None})$ 
   $\text{ObPosMap.Write}(\text{addr}, \text{leaf})$ 
   $\text{NobPosMap.Write}(\text{addr}, \text{leaf})$ 

 $\text{Write}(\text{addr}, b)$ 
 $\text{leaf} \leftarrow \text{NobPosMap.Read}(\text{addr})$ 
 $\text{DataTree.Write}(\text{leaf}, b)$ 

 $\text{Read}(\text{addr})$ 
 $\text{leaf} \leftarrow \text{ObPosMap.Read}(\text{addr})$ 
 $b \leftarrow \text{DataTree.Read}(\text{leaf})$ 
 $\text{leaf}' \leftarrow \text{Alloc}()$ 
 $\text{DataTree.Write}(\text{leaf}', b)$ 
 $\text{ObPosMap.Write}(\text{addr}, \text{leaf}')$ 
 $\text{NobPosMap.Write}(\text{addr}, \text{leaf}')$ 
return  $b$ 
```

---

we present wpORAM slightly differently, using *black-box* implementations of ORAM, OSAM and a read-public ORAM. While this helps abstract out the underlying details, each of our building blocks are also modifications of Path ORAM and work exactly as described in the overview.

**wpORAM’s components.** We use Path ORAM [29] to instantiate ORAM and Path OSAM [24] to instantiate OSAM. We instantiate the read-public ORAM using rpORAM<sup>7</sup>, a construction that we present in Section 6. As mentioned in Section 4, we will use the ORAM as an oblivious position map, the read-public ORAM as a non-oblivious position map, and OSAM for the data tree. Each value will be stored in the data ORAM at a distinct allocated SAM address, which can be thought of as a uniformly random “leaf” of the Path ORAM’s binary tree (indeed, this is how Path OSAM allocates them). Our position maps store mappings between addresses and their SAM addresses/leaves.

**Initializing Server Memory.** Before using wpORAM, the client runs an *Initialize* procedure that maps each address to a freshly allocated SAM address/leaf that is kept hidden from the server. The client stores these mappings in both position maps, and initializes the data-tree by writing  $\perp$  to each mapped SAM address/leaf.

**Implementing Writes for wpORAM.** To write a value  $v$  to an address  $\text{addr}$ , the wpORAM client looks up the non-oblivious position map to fetch the SAM address/leaf mapped to  $\text{addr}$ . The client then simply writes  $v$  to that leaf in the data tree. Note that, by the guarantees of read-public ORAM and OSAM, this does not leak the mapped SAM address/leaf to the server; while the read-public ORAM reveals which

address is being read, it does not reveal the contents of the value stored at that address.

**Implementing Reads for wpORAM.** To read from address  $\text{addr}$ , the wpORAM client reads the recursive oblivious position map to find the mapped SAM address/leaf, and then it reads that leaf from the data tree to fetch the value  $v$  written to  $\text{addr}$ . Since this leaf is read, due to SAM’s single-read constraint, the client writes  $v$  to a freshly allocated SAM address/leaf in the data tree. This ensures that each SAM address is read at most once, respecting SAM’s single access constraint. The client also updates both position maps with the new address-to-leaf mapping. By the guarantees of ORAM, OSAM and read-public ORAM, the only thing that the server learns is the fact that a read occurred.

**Theorem 1.** *Algorithm 1 implements a write-public ORAM (Definition 2), provided DataTree, ObPosMap and NobPosMap are implemented using OSAM, ORAM and read-public ORAM.*

*Proof.* First, notice that wpORAM is a valid SAM program, meaning that each allocated SAM address is read at most once. Observe that SAM addresses are read only during Reads, according to the mapping stored in ObPosMap. This mapping is initialized during the *Initialize* procedure, when the wpORAM client allocates and maps each address to a fresh SAM address. Subsequently, each time the wpORAM client performs a *Read* for an address  $\text{addr}$ , looks up ObPosMap to fetch the SAM address  $\text{leaf}$  mapped to  $\text{addr}$ , and reads  $\text{leaf}$  from DataTree. Then, the wpORAM client immediately allocates a *fresh* SAM address  $\text{leaf}'$  for  $\text{addr}$  and updates ObPosMap to store  $\text{leaf}'$  at  $\text{addr}$ . Hence, no SAM address is read twice, and wpORAM never halts.

We next show that wpORAM is correct, meaning that for any sequence  $\vec{r}$  of RAM requests, wpORAM implements the RAM interface for  $\vec{r}$ . That is, each *Read*( $\text{addr}$ ) request in  $\vec{r}$  returns the value  $v$  used in the *previous* *Write*( $\text{addr}, v$ ) request in  $\vec{r}$ , if any, and otherwise returns *None*. For this, consider two cases.

- *Case 1: There is no previous Write(addr, v) request in  $\vec{r}$ .* During the *Initialize* procedure, the wpORAM client maps  $\text{addr}$  to a SAM address  $\text{leaf}$ , and writes *None* to  $\text{leaf}$ . The wpORAM client also stores this mapping in ObPosMap by writing  $\text{leaf}$  to  $\text{addr}$  in ObPosMap. Since ObPosMap implements RAM, when the wpORAM client reads  $\text{addr}$  in ObPosMap during the *Read*, ObPosMap returns  $\text{leaf}$ . The wpORAM client then reads  $\text{leaf}$  in DataTree. Since DataTree implements SAM, it returns *None*.
- *Case 2: There is a previous Write(addr, v) request in  $\vec{r}$ .* During *Write*( $\text{addr}, b$ ), the wpORAM client reads NobPosMap to fetch the SAM address  $\text{leaf}$  mapped to  $\text{addr}$ , and writes  $v$  to  $\text{leaf}$  in DataTree. Notice that

---

<sup>7</sup>Pronounced as “roop ORAM.”

NobPosMap and ObPosMap always map  $\text{addr}$  to the same SAM address. This is because each time  $\text{addr}$  is mapped to a new SAM address  $\text{leaf}'$  during  $\text{Read}(\text{addr})$ , the wpORAM client writes  $\text{leaf}'$  to  $\text{addr}$  in *both* NobPosMap and ObPosMap. Since both position maps implement RAM, when the wpORAM client reads  $\text{addr}$  in ObPosMap, it fetches  $\text{leaf}$ . The wpORAM client then reads  $\text{leaf}$  from DataTree. Since DataTree implements SAM, it returns  $v$ .

It remains to show that wpORAM is a write-public ORAM which hides the access pattern during reads. For this, we construct a simulator  $\mathcal{S}$  that uses  $\mathcal{L}_{\text{wpORAM}}(\vec{r})$  (whether each request in  $\vec{r}$  is a Read/Write request, and the addresses used in Write requests) to simulate the server's view. Since the data tree, oblivious position map and non-oblivious position are instantiated using OSAM, ORAM and read-public ORAM, these structures have their respective simulators,  $\mathcal{S}_{\text{DT}}$ ,  $\mathcal{S}_{\text{ObPM}}$  and  $\mathcal{S}_{\text{NobPM}}$ , which we use to construct  $\mathcal{S}$ .

Recall from Definitions 2 and 4 that, to simulate the server's view, (1)  $\mathcal{S}_{\text{DT}}$  requires the number  $|\vec{r}_{\text{DT}}|$  of SAM requests sent to the data tree (2)  $\mathcal{S}_{\text{ObPM}}$  requires the number  $|\vec{r}_{\text{ObPM}}|$  of RAM requests sent to the oblivious position map and (3)  $\mathcal{S}_{\text{NobPM}}$  needs to know *which* requests are Read requests in the sequence of RAM requests  $\vec{r}_{\text{NobPM}}$  sent to the non-oblivious position map, including the addresses used in the Read requests (we denote this by  $\mathcal{L}_{\text{rpORAM}}(\vec{r}_{\text{NobPM}})$ ).

Since  $\mathcal{S}$  only uses  $\mathcal{L}_{\text{wpORAM}}(\vec{r})$  as input, we need to be able to compute  $|\vec{r}_{\text{DT}}|$ ,  $|\vec{r}_{\text{ObPM}}|$  and  $\mathcal{L}_{\text{rpORAM}}(\vec{r}_{\text{NobPM}})$  using  $\mathcal{L}_{\text{wpORAM}}(\vec{r})$ . This is straightforward for the first two, since  $\mathcal{L}_{\text{wpORAM}}(\vec{r})$  leaks the type of each request in  $\vec{r}$ , and each of these results in a deterministic number of requests sent by the wpORAM client to DataTree and ObPosMap. Notice that it is also easy to compute  $\mathcal{L}_{\text{rpORAM}}(\vec{r}_{\text{NobPM}})$  using  $\mathcal{L}_{\text{wpORAM}}(\vec{r})$  as they are essentially almost the *same*:  $\mathcal{L}_{\text{wpORAM}}(\vec{r})$  leaks the addresses used in Write requests in  $\vec{r}$ , and each of these results in a Read request to the *same* address in NobPosMap. The other requests made to NobPosMap are Write requests, and their order in  $\vec{r}_{\text{NobPM}}$  can be determined based on when Read requests occur in  $\vec{r}$ .  $\square$

## 5.1 wpORAM with Faster Initialization

The Initialize procedure in Algorithm 1 is as costly as an ORAM access, because it requires updating the oblivious position map with the initial mapping. Algorithm 2 presents a faster Initialize procedure that instead uses a PRF to determine the initial mapping and updates later mappings during reads. This avoids the need to initialize the position maps, and reduces bandwidth and roundtrip costs during Initialize by an  $O(\log N)$  factor.

**Initializing Server Memory.** Our faster Initialize procedure simply initializes all components of wpORAM — the data tree, oblivious position map and non-oblivious position map

---

### Algorithm 2: wpORAM client with cheaper Initialize

---

```

Initialize
  DataTree =  $\perp$ ; ObPosMap =  $\perp$ ; NobPosMap =  $\perp$ 
  // Write  $\perp$  to all buckets of DataTree,
  ObPosMap and NobPosMap

Write(addr, b)
  leaf  $\leftarrow$  NobPosMap.Read(addr) // If addr is
  not found in any bucket, returns  $\perp$ 
  if leaf ==  $\perp$  then leaf =  $F_k(\text{addr})$ 
  DataTree.Write(leaf, b)

Read(addr)
  leaf  $\leftarrow$  ObPosMap.Read(addr) // If addr not
  found during recursion level, reads
  random paths for remaining recursive
  calls and returns  $\perp$ 
  if leaf ==  $\perp$  then
    leaf =  $F_k(\text{addr})$ 
  ... // The remaining follows Algorithm 1.

```

---

— with  $\perp$ . In this case, initially, there are no address-to-SAM address mappings that can be used during reads and writes.

**Implementing Writes.** Since the position maps no longer store any initial mapping, the non-oblivious position map may return  $\perp$  while writing to an address. In this case, the client assumes a *default* address-to-SAM address mapping, determined using a PRF, that is used only until the first time the address is read. That is, the client uses  $F_k(\text{addr})$  to compute the mapped SAM address. If the non-oblivious position map does not return  $\perp$ , then the client uses the SAM address returned by the non-oblivious position map.

**Implementing Reads.** If the oblivious position map returns  $\perp$  when an address is read, then the client learns that this is the first time the address is being read. In this case, the client uses the PRF to determine the mapped SAM address. Otherwise, the client uses the SAM address returned by the oblivious position map. The client then performs the read as done in wpORAM. This includes updating the oblivious and non-oblivious position maps to store a (new) mapping from the address to a (freshly allocated) SAM address. This does not increase asymptotic cost since Read must access the oblivious position map anyway.

We remark that care must be taken while reading the oblivious position map. If, during any of the  $\log N$  recursive calls, some recursive call returns  $\perp$ , the remaining recursive calls must continue to read random paths from each tree. This should be done to prevent the server from learning that an address is being read for the first time.

---

**Algorithm 3:** Code run by the rpORAM client

---

```
Initialize
Stash  $\leftarrow \{\}$ 
for  $\text{addr} \in [0, N]$  do
   $\perp$  Write None to leaf  $\text{addr}$ .

Write( $\text{addr}, b$ )
Stash.Write( $\{b, \text{addr}\}$ )
Evict() // Performed according to Path OSAM

Read( $\text{addr}$ )
 $b \leftarrow \text{ReadPath}(\text{addr})$  // Fetch the latest
  block written to  $\text{addr}$  or None if there is
  no such block
return  $b$ 
```

---

## 6 Construction of rpORAM

Algorithm 3 presents our read-public ORAM construction rpORAM, which is a modification of Path ORAM that is inspired by the Path OSAM construction of [24]. We had presented an overview of rpORAM in Section 3. Here, we present the details.

**rpORAM’s components.** rpORAM is simply the data tree of Path ORAM without the position map. Each address  $\text{addr}$  is mapped to the leaf  $\text{addr}$ ; this is a *static* mapping that does not change. Looking ahead, this static mapping suffices because it is alright if Read operations repeatedly access the same path, as the addresses being read are public.

**Initializing Server Memory.** The client initializes server memory by writing None to each address  $\text{addr}$ . The client does this by writing None to each leaf of the data tree. The client also locally initializes a data-structure called the *stash*. Since buckets in the data-tree are of a fixed constant size in Path ORAM, not all data blocks may be able to fit in the tree. Path ORAM shows that if the client locally stores data blocks that “overflow” in the stash, then the probability that the stash exceeds size  $R$  decreases exponentially in  $R$ . Hence, it suffices for the client to have small local storage.

**Implementing Writes for rpORAM.** The client writes a block  $b$  to address  $\text{addr}$  by writing  $b$ , along with  $\text{addr}$ , to the client’s stash. In the overview, for ease of explanation, we had mentioned that the client writes to the data-tree’s *root*. In some sense, this is not entirely false, since the stash can be seen as an extension of the tree’s root.

After writing  $b$  to the stash, the client runs Evict to perform an eviction. During Evict, the client downloads a deterministic path from the server, and locally moves data-blocks on that path towards their assigned leaves. The client does this greedily by filling blocks from leaf to root (and stash), according to how much each bucket can hold. The client then uploads this path to the server.

Evict performed here follows that of *Path OSAM*, and is slightly different from what is done in Path ORAM. Since we allow “multiple” writes to the same address, a path to an address  $\text{addr}$  may have *outdated* versions of the value written to  $\text{addr}$ . During Evict, the client additionally deletes data-blocks with outdated versions of these values and only keeps the *latest* version of  $v$ , which is the one “closest” to the root.

The path that is evicted during each write is determined in a deterministic order. This ensures that the path evicted during each write, and the locations that the server accesses, are *independent* of the address that is being written to. Hence, writes are private and hide the addresses being written to.

While it is relatively straightforward to see that writes are private, it is harder to see why *correctness* holds. This is because Path ORAM’s bound on stash overflow uses a Chernoff bound that holds only when the mapped leaf that is chosen during each access is chosen uniformly at random, independent of other chosen leaves. This is not the case when multiple writes to the same address are allowed. Only recently, the Path OSAM work [24] gave a proof that bounds stash size even when multiple writes are allowed. However, their proof is involved, and we refer the interested reader to their paper for more details.

**Implementing Reads for rpORAM.** Recall that Reads are public, and reveal the address being read. Also recall that each address  $\text{addr}$  is statically mapped to the leaf  $\text{addr}$ . rpORAM follows Path ORAM’s invariant that the value written to an address lies somewhere on the path to the address’s mapped leaf. To read address  $\text{addr}$ , the client simply downloads the path to  $\text{addr}$  in the clear from the server, and locally searches this path to find the *latest* version of the value  $v$  written to  $\text{addr}$  (which is the one closest to the root). All of this is done by the ReadPath( $\text{addr}$ ) operation.

**Theorem 2.** Algorithm 3 implements a read-public ORAM (Definition 2).

*Proof.* Consider any sequence  $\vec{r}$  of RAM requests. We first show that each Read( $\text{addr}$ ) request in  $\vec{r}$  returns the value  $v$  used in the *previous* Write( $\text{addr}, v$ ) request in  $\vec{r}$ , if any, and otherwise returns None. Consider two cases.

- *Case 1:* There is no previous Write( $\text{addr}, v$ ) request in  $\vec{r}$ . During the Initialize procedure, the rpORAM client writes None to  $\text{addr}$ . During ReadPath( $\text{addr}$ ), the client downloads the path to  $\text{addr}$ , sees that there is no block written to  $\text{addr}$ , and outputs None.
- *Case 2:* There is a previous Write( $\text{addr}, v$ ) request in  $\vec{r}$ . During Write( $\text{addr}, v$ ), the client writes  $\{b, \text{addr}\}$  to the stash and performs periodic evictions to move  $b$  towards the leaf  $\text{addr}$ . By Path ORAM’s invariant,  $v$  always remains on the path to  $\text{addr}$ , and is the closest block to the root written to  $\text{addr}$ . Hence, during Read( $\text{addr}$ ), ReadPath returns  $v$ , and the client outputs  $v$ .

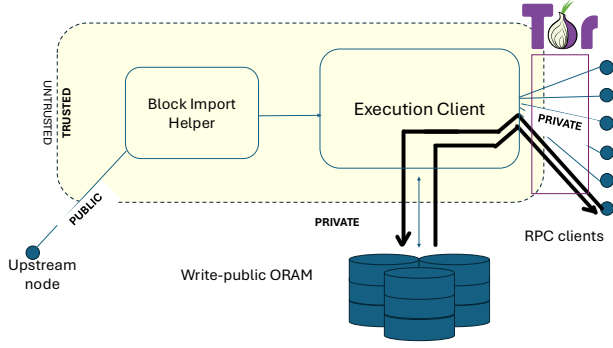


Figure 2: The architecture of our private RPC application implemented using wpORAM.

Further, by Theorem 1 of [24], the probability that the stash-size exceeds  $R$  decreases exponentially in  $R$ .

We next prove that rpORAM is a *read-public ORAM* by constructing a simulator  $\mathcal{S}$  that outputs  $\vec{r}'$  using  $\mathcal{L}(\vec{r})$ , where  $\mathcal{L}(\vec{r})$  is the sequence of Read operations in  $\vec{r}$ , along with the addresses being read. For each Write operation in  $\vec{r}$ ,  $\mathcal{S}$  downloads a deterministic (encrypted) path in a reverse-lexicographic order, and uploads another (re-encrypted) path. For each Read operation in  $\vec{r}$ ,  $\mathcal{S}$  knows which address  $\text{addr}$  is being read, and simply downloads the path to the leaf  $\text{addr}$ .  $\square$

## 7 Designing a Private Blockchain RPC Server

In this section, we describe how we use wpORAM to implement a private RPC server and evaluate its performance. We make the source code for our implementation publicly available at <https://github.com/trate3/write-public-oram-rpc>.

### 7.1 Architecture

We illustrate our private RPC architecture in Figure 2. We implement our RPC node as an Ethereum execution client (here on referred to as the *client*), with wpORAM as the backend database. We implement our own client as (i) most existing clients are tightly integrated with their database structures, and (ii) we wanted greater control on database update schedule. Our implementation passes the official Ethereum execution test suite [16]. Our client can run within a TEE (though we do not use a TEE in our benchmarks) and interact with untrusted external storage using wpORAM.

**Initial state sync.** We initialize the client database by enumerating the Ethereum state from a trusted node and invoking the Initialize procedure described in Section 5.1.

**The backend database.** Ethereum’s state consists of several object types that can be accessed separately: *accounts*, containing the balance, smart contract code hash, and transaction

nonce of each blockchain address; *storage*, containing the nonvolatile, non-empty storage of smart contracts; and *code*, which contains the bytecode of smart contracts.

We place each of these into separate wpORAMs. The size of values in each wpORAM’s data tree varies according to what each wpORAM stores: 104 bytes for accounts, 32 bytes for storage and 32 KB for code.

**Importing blocks.** Upon syncing, we import new blocks using a block import helper (also within the trust boundary of the TEE). The block import helper takes new blocks as input, computes the state difference with respect the previous state and hands the state updates to the client. The client then applies these changes to the database using the Write procedure of wpORAM.

**Serving RPC Requests.** Our client serves private read-only RPC requests by issuing oblivious read requests to wpORAM. Among the nontrivial RPC methods, our implementation supports `eth_getBalance`, `eth_getTransactionCount`, `eth_getCode`, `eth_getStorageAt`, `eth_call`, and `eth_estimateGas`. We also serve recent blocks and transactions from memory—though these can easily be extended to storage—to support the `eth_getBlock` and `eth_getTransaction` methods. However, a TEE host’s networking interface is also untrusted; an adversary on the network that is able to profile connections could deanonymize users through access patterns in network-level metadata. We therefore require users to hide such information by relaying their connections over an anonymity network like Tor.

### 7.2 Design Details

**Keyword ORAM.** As Ethereum addresses are 256 bits long, while wpORAM addresses are only  $\log N \ll 256$  bits, we store the value at Ethereum address  $a$  at  $H(a)$  in the oblivious position maps of wpORAM and Path ORAM, for hash function  $H$  with  $\log N$  bit outputs. By the standard balls-into-bins analysis, this results in mapping up to  $\frac{\log N}{\log \log N}$  (approximately 5 or 6 for our parameters) Ethereum address to a single ORAM address. Looking ahead, wpORAM easily handles this collision as each data block in wpORAM’s oblivious position map can store up to 64 values.

We remark that this problem is specific to the oblivious position map. Recall that the oblivious position map repeatedly remaps the address to a new leaf each time the address is read. Due to the need to handle collisions, this requires remapping all colliding addresses together. The non-oblivious position map does not face this issue: since reads are public, there is no need to re-map an address to a new leaf when it is read.

**Batching.** In both wpORAM and the baseline, we use the batching techniques of [12]. In particular, we combine database lookups across independent queries and use kernel-level asynchronous I/O to process them concurrently when their accesses do not overlap in the ORAM tree.

**Handling `eth_call` RPC requests.** The RPC function `eth_call` simulates a transaction on top of the current blockchain state. This simulation often involves multiple dependent reads from a consistent state. Therefore, while serving `eth_call` requests, we block all updates until the RPC call finishes to avoid generating a response based on an inconsistent database. In Section 9, we present a more efficient approach to handle serve `eth_call` requests without blocking the ORAM updates.

## 8 Evaluation

With our evaluation, we seek to show that wpORAM improves write performance over Path ORAM at a small cost to reads, and that this gain carries over to our private RPC server. We use Path ORAM as our baseline throughout.

**Experimental setup.** We run all experiments on a single non-TEE machine with an AMD Ryzen 9 9950X3D 16-core processor, 256 GB of memory, and a Samsung 9100 PRO 8 TB SSD. We store the entire database in external storage.

**ORAM setup.** Recall that wpORAM stores values in data-blocks, and data-blocks in nodes (buckets) of a tree. In our implementation, each node (or bucket) has the capacity to store up to 4 data-blocks, and each data-block stores up to 64 values. The value 64 lowers the number of recursions needed for querying the oblivious position map. When we compare our performance with that of Path ORAM, this only gives Path ORAM an advantage since we instantiate Path ORAM with the same parameters.

### 8.1 ORAM-Specific Evaluation

We first evaluate a keyword ORAM instantiated with wpORAM and Path ORAM, independent of the private RPC application. For these evaluations, we use 32 byte long database entries. For each database size tested, we first populate the database to half capacity using random keys and values to ensure the database is not sparsely filled.

**Throughput.** We first evaluate the throughput, the number of read and write operations that our implementation handles per second. We report throughput against database size in Figure 3, and its ratio to Path ORAM in Figure 4. Throughput falls off sharply beyond a database size of  $2^{28}$ , where the database no longer fits in the system file cache. Our improvement in write throughput over Path ORAM grows with the database size, and reaches  $3.52\times$  at  $2^{31}$ . Our read throughput is about 77% of Path ORAM’s across all database sizes, since our reads queries two position maps instead of one position map as in Path ORAM.

**Bandwidth.** Next, we measure bandwidth, which we define as the total number of bytes fetched from and written to the external storage, and we report it in Figure 5. Observe that, our improvements over Path ORAM grows with the database size,

| $N$                | 28    | 29    | 30    | 31    |
|--------------------|-------|-------|-------|-------|
| Total size on disk | 329GB | 660GB | 1.3TB | 2.6TB |

Table 2: Blockchain database sizes tested, each one consisting of three ORAM trees, with the storage tree having capacity of  $2^N$ , account tree having capacity of  $2^N/16$ , and code tree having capacity of  $2^N/256$ . Size on disk represents the total size of all three trees combined. For each on-disk size, 49.95% is occupied by the storage tree, 43.85% by the code tree, and 6.20% by the account tree.

and is about  $20\times$  for a database of size  $2^{31}$ . This gain comes from our writes never touching the oblivious position map, which is where the keyword ORAM of Section 7.2 incurs its blowup. Our read bandwidth is only slightly worse than Path ORAM’s bandwidth usage at every database size.

**Roundtrips.** Our improvement in roundtrips follows from the protocol description in Algorithm 1. Our writes take only two rounds irrespective of the database size, and our reads take the same number of rounds as Path ORAM.

### 8.2 Blockchain-Specific Evaluation

We next present performance numbers on our private RPC application. In each evaluation, we create a new blockchain database (using three ORAM trees for accounts, code, and storage, respectively). Our target upstream blockchain is Ethereum’s Hoodi testnet. We test four different database sizes, as shown in Table 2. The relative capacity of the three tries are approximately proportional to Hoodi’s first 1.5 million blocks’ usage; a storage tree of capacity  $2^{28}$ , account tree of capacity  $2^{24}$ , and code tree of capacity  $2^{20}$  fit the first 1.5 million blocks (224 days).

**State initialization.** Using the faster initialization algorithm introduced in Section 5.1, we measure the performance of initializing our ORAM database with blockchain state from a fixed block height. As shown in Figure 6, we find that our faster initialization method speeds up initial state synchronization, achieving an account initialization rate of more than double our standard write throughput.

**Block import speed.** After our oblivious blockchain database is synchronized with a certain block height, it must then catch up and continually import new blocks as they are added to the blockchain. We review the speed of writing new block data in units of gas, as individual blocks and blockchains have different actual gas usage. As shown in Figure 7, at a storage tree capacity of  $2^{31}$ , our oblivious RPC server can import new blocks at a speed of 259.5 Mgas/second. For comparison, Ethereum operates at approximately 2.5 Mgas/second on average [15].

**RPC call throughput.** We next measure in Figure 8 the throughput of `eth_call` RPC requests for a particular rep-

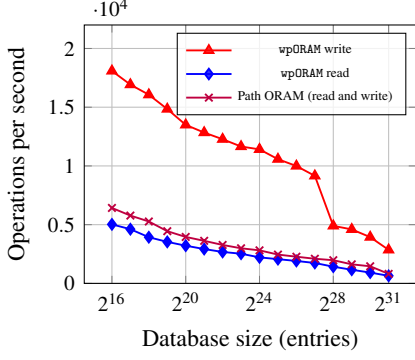


Figure 3: Read/write throughput for 32-byte values, with a sharp drop at  $2^{28}$  when the database exceeds the file cache.

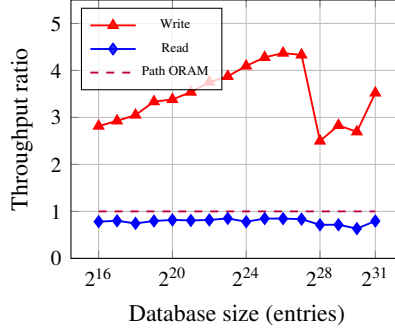


Figure 4: The ratio of wpORAM's read-write throughput to Path ORAM's throughput.

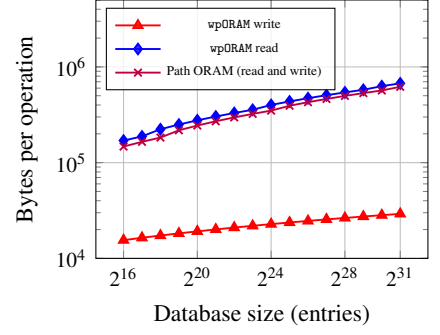


Figure 5: Bandwidth usage of wpORAM and Path ORAM, measured in number bytes per operation.

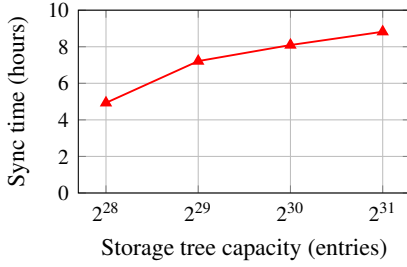


Figure 6: Time taken to perform the initial synchronization of 1.5M blocks.

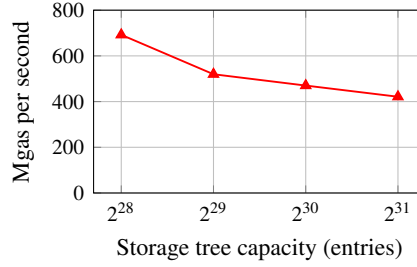


Figure 7: Block import speed (in million gas/second) after initial synchronization.

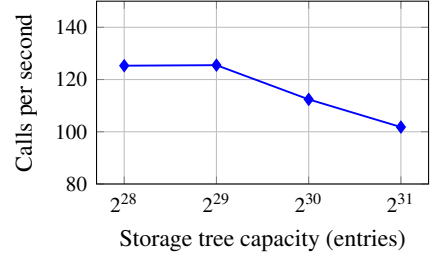


Figure 8: Throughput of ERC-20 balance lookup `eth_call` RPC requests.

representative call: fetching the ERC-20 token balance of an address; a user wanting to know his or her USDC balance would make a similar RPC call. These requests touch all three trees: two lookups to the account tree (sender and the contract address), one to the code tree (for the ERC-20 contract), and one to the storage tree (the user's balance). At our largest database sizes, a storage tree capacity of  $2^{31}$ , we were able to serve over 100 of these RPC calls per second.

## 9 Additional Optimizations and Features

We next describe optimizations and features that we do not implement, but that would improve the performance and reliability of a private RPC server.

**Batching state updates across blocks.** Despite wpORAM's improvement to write performance, fast blockchains with sub-second block times remain a challenge for a private RPC server. For such fast chains, clients can often tolerate receiving marginally older blockchain state, and we can take advantage of this tolerance. Rather than keeping the server perfectly up to date, we allow responses to use state as old as a maximum delay  $\Delta$ , say  $\Delta = 1$  second. The server then applies the state changes of every block in a batch spanning total time  $\Delta$  at once. In batching blockchain state changes, state slots that

were modified in more than one block are compressed to a single update. This reduces the number of total writes for popular state locations.

To estimate the benefit, we sampled groups of contiguous Ethereum blocks between heights 23.5M and 25.5M, and compared the state differences of the individual blocks against the aggregate difference across each group. Figure 9 reports the results. The reduction is modest, and it approaches a logarithmic trend at larger batch sizes.

**Improved handling of `eth_call` requests.** In Section 7, we kept each `eth_call` RPC request consistent with the blockchain state by blocking all state updates until the request completes. A long `eth_call` with hundreds of reads consequently adds latency to all other requests.

Instead of writing every block's state changes to the database immediately, a different approach is to maintain an in-memory ring buffer containing the state differences of a constant number of recent blocks. When a new block arrives, we remove the oldest state changes from the buffer, write them to the database, and then add the new block's state changes to the end of the ring buffer. These data structures should be implemented using ORAM.

When an RPC call needs to read from the database, it reads a key  $k$  with respect to the state at a particular block height.

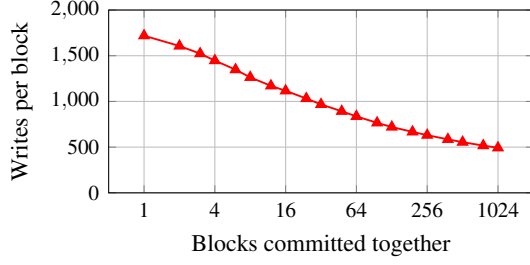


Figure 9: Average number of writes per block when committing the state changes of multiple Ethereum blocks together. State modified in more than one block of a batch requires a single write, so larger batches incur fewer writes per block. Each data point is the mean of a random sample of 250 batches from Ethereum mainnet blocks 23.5M to 25.5M.

For each read, then, we walk through the ring buffer from the newest changes to the oldest changes. If we discover a change that is both (1) at least as old as our target block height and (2) modifies  $k$ , we have discovered the target value for  $k$  at the correct height, so we return it. If walking through the entire ring buffer did not find such a change, then we must query the database. (Even if we do not need to query the database, we nevertheless issue a dummy request to prevent leaking whether the request was for recently changed state.) After reading an item from the database, we verify that the request was not for state older than the oldest block in the buffer. If the request is for block state at a height older than the last block popped from the buffer, we cannot guarantee that the target state was not overwritten in a more recent block, so we cause the RPC request to fail.

In addition to enabling new blocks to be applied in the middle of a multi-read RPC request, this improvement allows our RPC server to support requests at recent block heights instead of only the latest block height.

**Verifiability.** Users today trust RPC servers for correctness, or otherwise operate their own nodes or light clients such as Helios [1] to verify blockchain state. Our implementation trusts two sources of data: the upstream Ethereum execution node it imports data from and the external storage the TEE reads back. We can authenticate both by relying on the integrity of the TEE. The upstream node can supply Merkle proofs against Ethereum’s state roots, which our TEE client would verify before applying the updates. For external storage, the Merkle tree technique from [25] can be applied to wpORAM.

## 10 Related Work

**General Purpose ORAMs.** Oblivious RAM (ORAM), introduced in [17], has witnessed significant progress over the years [6, 7, 18, 19, 20, 26, 28, 29, 32]. Modern ORAM constructions are now nearly optimal. It is known that any

ORAM must incur an  $\Omega(\log N)$  bandwidth overhead [21]. Path ORAM [29] achieves this overhead for block sizes of  $\Omega(\log^2 N)$  and remains the most practical ORAM construction to date. Meanwhile, OptORAMa [7] attains the same asymptotically optimal overhead for blocks of arbitrary size, but with substantially larger concrete costs. Despite these advances, both Path ORAM and OptORAMa require  $O(\log N)$  round-trips per access.

**Special Purpose ORAMs.** Because general purpose ORAMs remain costly in practice, recent works [9, 22, 27, 31] relax the guarantees of ORAM to reduce costs. One work [31] studies read-only ORAMs for the special case when, after the database is initialized, *only read* requests are issued. Our goal is to construct an ORAM that instead allows *both* reads and writes, but only *hides* the addresses read during *reads*.

Another line of work [9, 22, 27] studies so-called write-only ORAMs. These ORAMs consider a setting in which the attacker can only see which *physical writes* were performed to the server’s memory, and cannot see which physical reads were performed. Such a threat model is natural when the adversary obtains only periodic snapshots of the server’s memory state. We instead consider a stronger setting in which the adversary may observe the protocol execution continuously, hence observing physical writes *and* reads.

**Oblivious Data Structures/Pointer Programs.** Prior work has also considered oblivious algorithms for special purpose RAM programs, including for tree-like data structures [33], pointer machine programs (including graphs) [4, 24] and immutable pointer machine programs [3]. Our work focuses on obfuscating the access pattern for *general-purpose* RAM programs, but while only keeping writes private.

## 11 Conclusion

In this paper, we presented write-public ORAM, a relaxation of ORAM for applications that perform private reads on public data. We designed wpORAM, a write-public ORAM that improves the amortized bandwidth and roundtrips per write by a  $\log N$  factor over Path ORAM, the most efficient ORAM in practice. Our evaluation on databases with up to  $2^{31}$  entries shows that this asymptotic gain translates to a 3–4 $\times$  improvement in write throughput, with reads only approximately 23% slower. We used wpORAM to implement an oblivious blockchain RPC server for the Ethereum Virtual Machine; to the best of our knowledge, ours is the first such design.

## References

- [1] a16z. *Helios Light Client*. Accessed: 2026-08-06. URL: <https://helios.a16zcrypto.com/>.

- [2] Sebastian Angel et al. “PIR with compressed queries and amortized query processing”. In: *2018 IEEE symposium on security and privacy (SP)*. IEEE. 2018, pp. 962–979.
- [3] Ananya Appan and David Heath. “Oblivious Immutable Memory”. In: *Cryptology ePrint Archive* (2025).
- [4] Ananya Appan, David Heath, and Ling Ren. “Oblivious single access machines-A new model for oblivious computation”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024, pp. 3080–3094.
- [5] Pranav Shriram Arunachalam, Yue Chen, and Ling Ren. “Private Information Retrieval: A Tutorial and Survey”. In: *Proceedings of the IEEE* (2026).
- [6] Gilad Asharov, Ilan Komargodski, and Yehuda Michelson. “FutORAM: A Concretely Efficient Hierarchical Oblivious RAM”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, pp. 3313–3327.
- [7] Gilad Asharov et al. “OptORAM: optimal oblivious RAM”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2020, pp. 403–432.
- [8] Ali Atiia and Keewoo Lee. *Sharded PIR Design for the Ethereum State*. <https://ethresear.ch/t/sharded-pir-design-for-the-ethereum-state/24552>. Ethereum Research forum post. Accessed: 2026-07-30. Mar. 2026.
- [9] Erik-Oliver Blass et al. “Toward robust hidden volumes using write-only oblivious ram”. In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 2014, pp. 203–214.
- [10] Quang Cao et al. “TreePIR: Efficient Private Retrieval of Merkle Proofs via Tree Colorings with Fast Indexing and Zero Storage Overhead”. In: *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2025, pp. 4476–4494.
- [11] ChainList. *ChainList: A List of EVM Networks*. Accessed: 2026-06-09. URL: <https://chainlist.org/>.
- [12] Anrin Chakraborti and Radu Sion. “ConcurORAM: High-throughput stateless parallel multi-client ORAM”. In: *arXiv preprint arXiv:1811.04366* (2018).
- [13] Yue Chen and Ling Ren. “OnionPIRv2: Efficient Single-Server PIR”. In: *Cryptology ePrint Archive* (2025).
- [14] Benny Chor et al. “Private information retrieval”. In: *Journal of the ACM (JACM)* 45.6 (1998), pp. 965–981.
- [15] Ethereum Foundation. *Blocks*. <https://ethereum.org/developers/docs/blocks/>.
- [16] Ethereum Foundation. *Ethereum Execution API Tests*. URL: <https://github.com/ethereum/execution-apis/blob/main/docs-api/docs/tests.md> (visited on 07/28/2026).
- [17] Oded Goldreich and Rafail Ostrovsky. “Software protection and simulation on oblivious RAMs”. In: *J. ACM* 43.3 (May 1996), pp. 431–473. ISSN: 0004-5411. DOI: 10.1145/233551.233553. URL: <https://doi.org/10.1145/233551.233553>.
- [18] Michael T Goodrich and Michael Mitzenmacher. “Privacy-preserving access of outsourced data via oblivious RAM simulation”. In: *International Colloquium on Automata, Languages, and Programming*. Springer. 2011, pp. 576–587.
- [19] Michael T Goodrich et al. “Privacy-preserving group data access via stateless oblivious RAM simulation”. In: *Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms*. SIAM. 2012, pp. 157–167.
- [20] Eyal Kushilevitz, Steve Lu, and Rafail Ostrovsky. “On the (in) security of hash-based oblivious RAM and a new balancing scheme”. In: *Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms*. SIAM. 2012, pp. 143–156.
- [21] Kasper Green Larsen and Jesper Buus Nielsen. “Yes, there is an oblivious RAM lower bound!” In: *Annual International Cryptology Conference*. Springer. 2018, pp. 523–542.
- [22] Lichun Li and Anwitaman Datta. “Write-only oblivious RAM-based privacy-preserved access of outsourced data”. In: *International Journal of Information Security* 16.1 (2017), pp. 23–42.
- [23] Samir Jordan Menon and David J Wu. “Spiral: Fast, high-rate single-server PIR via FHE composition”. In: *2022 IEEE symposium on security and privacy (SP)*. IEEE. 2022, pp. 930–947.
- [24] Sage Pia et al. “Oblivious Single Access Machines are Concretely Efficient”. In: *Cryptology ePrint Archive* (2026). URL: <https://eprint.iacr.org/2026/451>.
- [25] Ling Ren et al. “Integrity verification for path oblivious-ram”. In: *2013 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE. 2013, pp. 1–6.
- [26] Ling Ren et al. “Ring oram: Closing the gap between small and large client storage oblivious ram.” In: *IACR Cryptol. ePrint Arch.* 2014 (2014), p. 997.

- [27] Daniel S Roche et al. “Deterministic, stash-free write-only oram”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2017, pp. 507–521.
- [28] Elaine Shi et al. “Oblivious RAM with  $O((\log N)^3)$  worst-case cost”. In: *International Conference on The Theory and Application of Cryptology and Information Security*. Springer. 2011, pp. 197–214.
- [29] Emil Stefanov et al. “Path ORAM: an extremely simple oblivious RAM protocol”. In: *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*. CCS ’13. Berlin, Germany: Association for Computing Machinery, 2013, pp. 299–310. ISBN: 9781450324779. DOI: [10.1145/2508859.2516660](https://doi.org/10.1145/2508859.2516660). URL: <https://doi.org/10.1145/2508859.2516660>.
- [30] Afonso Tinoco et al. *Scalable Oblivious Accesses to Blockchain Data*. <https://writings.flashbots.net/scalable-oblivious-accesses-to-blockchain-data>. Flashbots Writings, June 2, 2025. Accessed: 2026-07-30. June 2025.
- [31] Shruti Tople, Yaoqi Jia, and Prateek Saxena. “{PRO-ORAM}: Practical {Read-Only} Oblivious {RAM}”. In: *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*. 2019, pp. 197–211.
- [32] Xiao Wang, Hubert Chan, and Elaine Shi. “Circuit oram: On tightness of the goldreich-ostrovsky lower bound”. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 2015, pp. 850–861.
- [33] Xiao Shaun Wang et al. “Oblivious data structures”. In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 2014, pp. 215–226.